

SimPhony: A Device-Circuit-Architecture Cross-Layer Modeling and Simulation Framework for Heterogeneous Electronic-Photonic AI System

Ziang Yin¹, Meng Zhang², Nicholas Gangi², Rena Huang², Jeff Zhang¹, Jiaqi Gu^{1†}

¹Arizona State University, ²Rensselaer Polytechnic Institute

†jiaqigu@asu.edu

Abstract— Electronic-photonic integrated circuits (EPICs) offer transformative potential for next-generation high-performance AI, but they require interdisciplinary advances across devices, circuits, architecture, and design automation. The complexity of these hybrid systems makes it challenging even for domain experts to understand distinct behaviors and interactions across the design stack. The lack of a flexible, accurate, fast, and easy-to-use EPIC AI system simulation framework significantly limits the exploration of hardware innovations and system evaluations on common benchmarks. To address this gap, we propose *SimPhony*, a cross-layer modeling and simulation framework for heterogeneous electronic-photonic AI systems. *SimPhony* offers a platform that enables (1) generic, extensible hardware topology representation that supports heterogeneous multi-core architectures with diverse photonic tensor core designs; (2) optics-specific dataflow modeling with unique multi-dimensional parallelism and reuse beyond spatial/temporal dimensions; (3) data-aware energy modeling with realistic device responses, layout-aware area estimation, link budget analysis, and bandwidth-adaptive memory modeling; and (4) seamless integration with model training framework for hardware/software co-simulation. By providing a unified, versatile, and high-fidelity simulation platform, *SimPhony* enables researchers to innovate and evaluate EPIC AI hardware across multiple domains, facilitating the next leap in emerging AI hardware. Our code is open-sourced at [link](https://github.com/ScopeX-ASU/SimPhony)¹.

I. INTRODUCTION

Heterogeneous electronic-photonic integrated circuits (EPICs) are emerging platforms for high-performance artificial intelligence (AI) computing. Demonstrated optical neural networks (ONNs) showcase breakthroughs in their performance and efficiency [1]–[9]. There is extensive research progress in materials, devices, and circuits aiming to overcome technological barriers in photonic AI. Some cross-layer co-design [10]–[14] and architecture optimization [4], [5], [15]–[19] efforts have scaled these systems for real-world AI tasks. Exploring the full stack of EPIC AI systems demands expertise across physics, devices, analog circuits, system architecture, electronic-photonic design automation (EPDA), and AI algorithms, as system-level evaluation is critical to understanding innovations at each individual design layer.

However, the complexity of such a hybrid system makes it challenging even for experts to grasp the behavior of each component and its interactions across software and hardware. Key challenges include: **❶ Lack of Unified Representation of Distinct PTC Circuit Topology:** Photonic tensor cores (PTCs) employ diverse optical principles for matrix computation, e.g., magnitude/phase modulation, wavelength/mode/time-division multiplexing (WDM, MDM, TDM), interference, diffraction, etc. Such abundant design

flexibility creates various PTC circuit topologies, e.g., weight bank [20], triangular mesh [1], [21], rectangular mesh [22], butterfly mesh [3], crossbar [2], [4], [17], single WDM link [23], etc. Prior simulators are modified from digital tools [24] that only support array-like computing architectures and cannot represent diverse PTC topologies. **❷ Lack of Support for Optics-Specific Dataflow and Parallelism:** EPIC AI systems involve parallelism and resource sharing beyond temporal/spatial dimensions. The combination of optical broadcasting, hierarchical accumulation, and multi-dimensional reuse patterns results in a complex dataflow. The additional optical dimensions, like magnitude/phase, wavelength, polarization, and modes, further complicate the design space. Thus, a flexible framework tailored to optics-specific needs is required. **❸ Lack of Accurate Model-Circuit-Layout Co-Modeling:** Unlike digital AI accelerators, analog systems integrate models tightly with devices/circuits, leading to inaccuracies in energy modeling due to a lack of real workload data and precise device settings. Beyond simple analytical models, there is a strong need to incorporate rigorous simulations and even chip measurements into energy analysis. Additionally, existing EPIC design tools overlook the layout when estimating the chip area. Previous methods [4], [25], [26] aggregate device footprints, leading to an underestimate of area. Therefore, a fast yet accurate layout estimator is essential for more reliable area analysis.

In this work, we present *SimPhony*, an open-source cross-layer modeling and simulation framework for heterogeneous EPIC AI systems. Built with a customized EPIC device library, *SimPhony* enables the hierarchical construction of heterogeneous photonic architectures from arbitrary PTC circuit topologies. Integrated with the ONN training library, *SimPhony* enables end-to-end simulation, including workload extraction, memory construction, and dataflow generation, while accurately analyzing system latency, data-aware energy, and layout-aware area.

Our main contributions are as follows,

- We introduce *SimPhony*, a cross-layer simulation framework for EPIC AI systems, enabling end-to-end accurate performance and efficiency analysis.
- **Unified PTC Representation:** Utilizing our customized device library, we design a hierarchical netlist-based circuit representation that generates arbitrary PTC topologies with automatically derived scaling rules and critical paths.
- **Photonics-Specific Dataflow Handling:** *SimPhony* accommodates multi-dimensional parallelism and hierarchical accumulation, effectively integrating the unique characteristics and dataflow of photonic computing hardware.

¹<https://github.com/ScopeX-ASU/SimPhony>

TABLE I: PTC taxonomy [8]: PTC designs with distinct properties in operand range and reconfiguration frequency (*Reconfig*) with different # of forward required to obtain full-range output.

EPIC Designs	Operand A		Operand B		Forward Method	#Forwards
	Range	Reconfig	Range	Reconfig		
MZI Array [1]	$\mathbb{R}$	Dynamic	$\mathbb{R}$	Static	Direct	1
Butterfly Mesh [10]	$\mathbb{R}$	Dynamic	$\mathbb{C}$	Static	Pos-Neg	1
MRR Array [20]	$\mathbb{R}^+$	Dynamic	$\mathbb{R}$	Dynamic	Direct	2
PCM crossbar [27]	$\mathbb{R}^+$	Dynamic	$\mathbb{R}^+$	Static	Direct	4
TeMPO [17]	$\mathbb{R}$	Dynamic	$\mathbb{R}$	Dynamic	Direct	1

- **Accurate System Modeling:** SimPhony provides accurate energy analysis based on real workload data and realistic device power models, along with accurate area analysis featuring auto-generated layout estimation.

II. PRELIMINARY

A. Photonic Tensor Core Taxonomy and Modeling Challenges

The diverse properties of PTC designs result in variations in circuit topology, devices, and operational principles, affecting speed and power. Table I categorizes PTCs by expressivity, computing mechanism, operand range, and reconfiguration speed [8]. For expressivity, universal PTCs can perform arbitrary matrix multiplication, e.g., micro-ring (MRR) arrays [20], while subspace PTCs support only a subset of static linear transformation [3]. Based on the numerical range of input operands, full-range PTCs compute arbitrary values in one shot, whereas subspace coherent PTCs (e.g., Butterfly meshes) require two differential computations for full-range input/output. Incoherent PTCs, e.g., MRR arrays [20], also require two computations to handle full-range inputs, while non-volatile phase change material (PCM) crossbar arrays [2] need up to four cycles. Reconfiguration speed is another key factor to determine its supported dataflow and operations. Thermo-optic MZI arrays require matrix decomposition and thermal tuning to reconfigure the weights, leading to a delay of μs to ms and limiting them to weight-stationary dataflows, unsuitable for dynamic self-attention. Dynamic PTCs [4], [17] use high-speed modulators for real-time matrix switching, enabling dynamic tensor products and output-static dataflows. Developing a modeling framework that accurately models complex system performance is challenging.

PTCs also feature varied topologies beyond conventional crossbars, such as triangular [21] and butterfly meshes [3]. Architectural features like optical broadcast, multi-dimensional sharing [4], analog-domain accumulation, and electrical-optical (E-O) interfaces further complicate system description and accurate modeling.

B. Photonic Accelerator Simulation Tools

Prior photonic AI hardware often relies on internal closed-source analytical device modeling and behavior-level modeling [3], [15], [16], [28], [29], unaware of the value in the actual workload. CimLoop is a recent analog NN simulator; its photonic version [24] has showcased one architecture Albireo [5], but it focuses on architectural parameters (e.g., memory and dataflow), and lacks flexible PTC construction with photonics-specific dataflow and parallelism. Its simplified device modeling and the complicated interface of the Timeloop-based framework make it challenging for device and circuit researchers to explore hardware optimization efficiently. Furthermore, previous efforts have not integrated ONN training at the application level, leaving no unified framework for

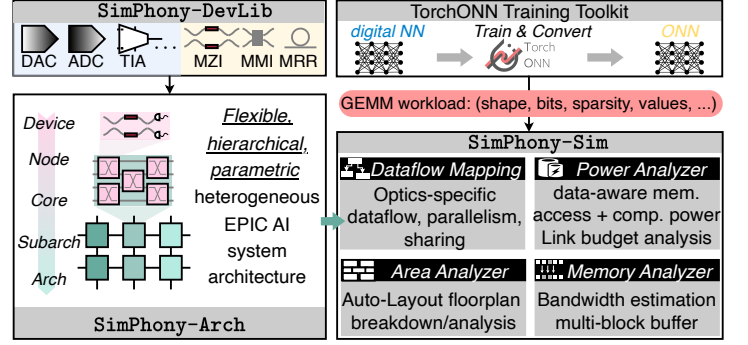


Fig. 1: Overview of our proposed SimPhony framework.

device/circuit-centric system modeling, which is critical for EPIC AI efficiency and performance modeling.

III. SIMPHONY: EPIC AI SYSTEM MODELING AND SIMULATION FRAMEWORK

Figure 1 summarizes the simulation flow and key components of our proposed SimPhony framework. It contains a customized electronic-photonic device library SimPhony-DevLib and supports flexible, hierarchical, and parametric modeling of EPIC AI system architecture SimPhony-Arch. Integrated with TorchONN model training toolkit, our simulation system SimPhony-Sim handles photonics-specific dataflow with bandwidth-adaptive memory hierarchy, data-aware power estimation, link budget analysis, and layout-aware chip area analysis.

A. SimPhony-DevLib: Comprehensive and Customizable Electronic-Photonic Device Library

One key novelty of our simulation framework is the support of a comprehensive and customizable device library SimPhony-DevLib, which lays the foundation for flexible architecture construction and accurate system modeling.

Modeling Granularity: SimPhony-DevLib device modeling is based on experimental data reported, ensuring an accurate representation of device characteristics. Specifically, the photonic device power models are obtained through Lumerical HEAT simulation or experimental measurements that are aware of actual device configuration, ensuring fast and accurate cost modeling.

Modeling Approach: We organize key components into electrical and optical categories, including a variety of high-performance photonic and electronic devices. Comprehensive device information is provided in detail to support accurate simulations of area, power, latency, and link budget. Devices from foundry PDKs can be plugged in. Our device library supports flexible scaling with different technology nodes, port numbers, working conditions, and more. For example, digital-to-analog converters (DACs) in our library support power scaling with customized sampling rates and bit resolutions, enabling power optimization via gating or quantization. On the photonic device front, e.g., the electro-optic Mach-Zehnder modulator (MZM) is widely used for high-speed encoding. To achieve precise performance modeling, we collect various properties such as spatial size, bandwidth, insertion loss, modulation efficiency, static power, extinction ratio, testing bitwidth, and more.

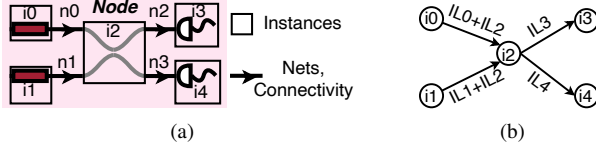


Fig. 2: (a) Node-level circuit topology definition and (b) corresponding weighted direct acyclic graph (DAG) representation.

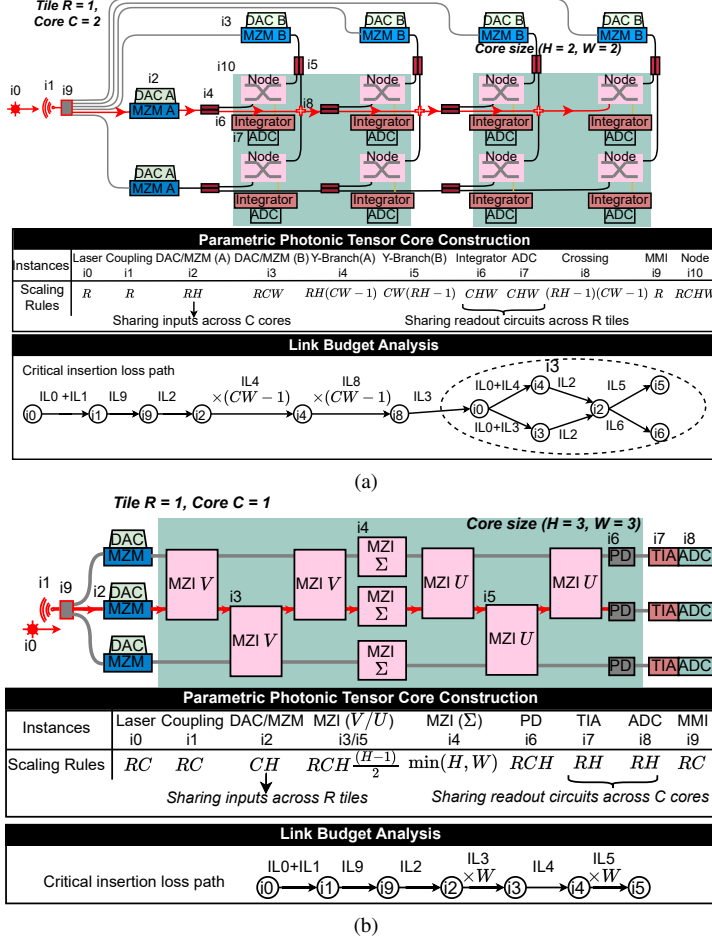


Fig. 3: Case studies of constructing parametric multi-core accelerator architecture (a) dynamic array-style TeMPO [17] and (b) static mesh-style MZI arrays [1]. The circuits can be generated by scaling up the smallest node with a user-defined scaling rule. The critical path with the highest insertion loss can be auto-derived.

B. SimPhony-Arch: Hierarchical, Parametric Heterogeneous EPIC AI System Architecture Builder

To enable flexible construction of heterogeneous multi-core architectures with diverse PTC designs and dataflows, we introduce SimPhony-Arch, a hierarchical, parametric architecture builder. Existing simulators focus on dataflow-centric architectural modeling with fixed PTCs, neglecting device/circuit details. This limits their usage for device/circuit customization, underscoring the need for a universal representation to unify diverse PTC designs.

To enable flexible PTC construction, we customize a *netlist* representation in SimPhony-Arch to describe devices as instances and port connectivity as directed 2-pin nets. Unlike electrical circuit netlists with undirected multi-pin nets, PTCs require directed 2-pin nets to capture the directional optical signal flow.

Key observations of PTC design patterns inspire us to use *modular circuit construction*, avoiding manual scripting of large netlists. This allows us to define a *minimal building block* denoted as *node*, e.g., a dot-product unit shown in Fig. 2(a), and *build the circuit according to specific scaling rules* with a user-defined node connection topology. A weighted directed acyclic graph (DAG) is generated based on the node topology, as shown in Fig. 2(b). The topology and insertion loss-based edge weights are essential in link budget analysis and layout-aware area estimation. This universal, hierarchical netlist interface also enables potential SPICE simulation and physical design as a future extension.

In the following, we provide two case studies of how to construct representative PTC architectures in a parametric style.

Case Study 1: Dynamic Array-style Tensor Cores TeMPO [4], [17]. The first case study demonstrates how to construct an array-style PTC architecture, TeMPO [4], [17], designed for dynamic time-multiplexed tensor products, as shown in Fig. 3(a). We first define the *architecture parameters*: R tiles, each containing C cores, with $H \times W$ dot-product nodes per core performing parallel computations. Then, we define the structure of the minimum building block, i.e., the dot-product unit, denoted as a *node*. A *node netlist* is used to describe the 6-device circuit topology using *directed* 2-pin nets to represent the waveguide connections and signal flow, shown in Fig. 2(a). To efficiently span the multi-core architecture without manually detailing every connection, we define *scaling rules* applied to each node and describe inter-node connections. This approach supports **parametric generation** of the architecture, enabling **automatic analysis** of area, power, and link budget. For example, the device area/power estimation engine will trace the netlist to count the number of devices considering *hardware sharing*. There are $RCHW$ total nodes for parallel dot-product. As the output of C cores in a tile are *in-situ* accumulated, integrators/ADCs can be shared and thus scaled by CHW . MZM group A encodes one matrix operand and can be broadcast to R tiles. Thus, the input encoders, i.e., DAC A and MZM A, are scaled by RH . These scaling rules are expressed as *customizable symbolic expressions* in circuit description files, enabling user-defined reuse styles to suit specific designs. The link budget analyzer will trace the netlist and auto-extract the longest path from the weighted DAG of the hierarchical netlist. Edge weights are automatically assigned to the insertion loss of incident vertices to match the parametric architecture settings, e.g., edge pointing to $i4$ stores $(CW-1) \times$ the loss of device $i4$.

Case Study 2: Static Mesh-style MZI Array [1], [22]. Besides array style, SimPhony-Arch can handle *more challenging mesh-style* PTCs, e.g., Clements-style MZI meshes in Fig. 3(b). The matrix is encoded by singular value decomposition ($U\Sigma V$), where unitary matrices U/V are decomposed to interleaved MZI meshes. To parametrically generate MZI meshes, we follow the *node* description to define a single-MZI node- U , node- Σ , and node- V . Our flexible *scaling rule* allows users to independently build unitary matrices by scaling node- U/V by $RCH(H-1)/2$ times and build diagonal by scaling node- Σ by $RC \min(H, W)$ times, which is *not representable by prior simulators that are based on arrays*. Details of the critical path and hardware sharing are omitted here, as they follow similar principles to Case Study 1.

C. SimPhony-Sim: EPIC AI System Simulation Flow

SimPhony-Sim is an end-to-end simulation flow, including NN model conversion and workload extraction to memory simulation and analysis of latency, area, and energy cost.

1) *ONN Model Conversion and Workload Extraction*: For digital DNN accelerator simulation, model training and hardware mapping are sufficiently decoupled. In contrast, analog mixed-signal EPIC AI systems require cross-layer co-design, resulting in closely coupled model training, conversion, mapping, and architecture simulation processes. Our SimPhony-Sim system can seamlessly interface with an open-source ONN training library, TorchONN [30], that supports various customized ONN types. A digital DNN will first be **converted to its analog optical version with layer-wise conversion**, e.g., Conv2d to TeMPOConv2d. The converted model will be trained with device non-ideality, quantization, pruning, and various co-design methodologies to ensure accuracy, robustness, and energy efficiency. The converted ONN model will be parsed by SimPhony-Sim to extract the detailed workload configuration for each layer, including input/weight size, input/weight/output bitwidth, pruning mask, scaling factors, **actual weight values**, etc. Weight values can have different modes, e.g., matrix values, normalized device transmissions, phase shifts, or even control voltages, which are useful for **precise value-aware power modeling**. Convolution, linear, and attention layers will be converted to general matrix multiplication (GEMM) representations. Other less computation-intensive layers are offloaded to electrical processors and omitted here for simplicity. With a layer-to-arch mapping configuration, we enable the flexibility to **map different layers to different types of sub-architectures** based on their compatibility and efficiency considerations, enabling heterogeneous computing paradigms.

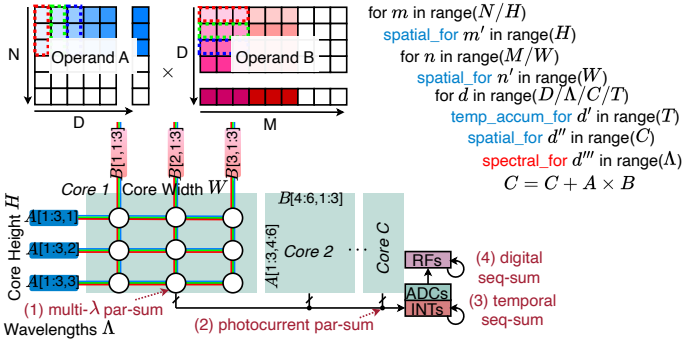


Fig. 4: Mapping blocking GEMM to output-stationary PTC with spatial/spectral parallelism and hierarchical accumulation.

2) *Photonics-Specific Dataflow and Latency Analysis*: Besides the support for standard dataflow for GEMM, e.g., weight/input/output stationary, here, we emphasize unique photonic-specific mapping and parallelism in SimPhony-Sim.

Multi-Dimension Parallelism and Hierarchical Accumulation. Beyond the spatial and temporal dimensions of electrical hardware, optical systems have more physical dimensions for parallel computing and hardware sharing, e.g., spectral, polarization, modes, etc. Figure 4 illustrates a partitioned GEMM workload mapped to a multi-core TeMPO architecture with multi-dimensional parallelism and hierarchical accumulation. As shown

in the nested loop representation, multiple wavelengths are used for spectral parallel summation, photocurrents from C cores are aggregated for analog-domain parallel summation, spatial parallelism is used for parallel outer product, temporal integration is used for analog sequential summation, and the partial sum is further sequentially accumulated digitally in the local buffer. Based on the mapping and parallelism, we will *derive the system execution latency in units of cycles*.

Latency Penalty for Range-Restricted PTCs. As highlighted in Section II and Table I, PTCs are constrained in their *numerical range representation due to device modulation features*, leading to *varying processing times to complete full-range computations*. We denote the number of iterations to acquire full-range output as I . Especially for PTCs that can only encode positive inputs/weights, four times the execution cost is required to realize full-range output, i.e., $I=4$. SimPhony will automatically analyze the tensor core property based on input/weight/output encoding properties and generate the corresponding dataflow with $I \times$ latency penalty.

PTC Reconfiguration Latency Penalty. In weight-stationary PTCs, loading new weights is sometimes bottlenecked by slow device reprogramming rather than memory loading. For instance, thermo-optic (TO) devices have a thermal time constant of ~ 10 μ s, and writing to phase change material (PCM) cells incurs a delay of over 100 ns. SimPhony-Sim automatically analyzes reprogramming latency and **applies corresponding cycle penalty whenever weight loading causes circuit reconfiguration delays exceeding one clock cycle**, e.g., 500 cycles per switch for 100 ns reconfiguration delay at 5 GHz. The total latency of mapping one layer is $\tau_{tot} = \tau_{load-input/weight} + \tau_{write-out} + I(\tau_{comp} + \tau_{reconfig})$.

3) *Bandwidth-Adaptive Memory Hierarchy Modeling*: One of the key points to enabling photonics advantage in AI computing is *sufficient data movement bandwidth and latency-hiding techniques*. Note that we focus on memory bandwidth analysis and assume on-chip and cross-chiplet interconnects provide sufficient data transaction bandwidth, especially when optical interconnects are available for multi-Tbps signal fanout/broadcast bandwidth [31]. SimPhony-Arch adopts a four-level memory hierarchy consisting of off-chip High Bandwidth Memory (HBM), Global Buffer (GLB), Local Buffer (LB), and Register File (RF). Each memory level stores operands A, B, and the output in progressively smaller sizes: i.e., the entire model at the HBM level, a single layer at the GLB level, the processing matrix dimensions at the LB level, and data for a single cycle at the RF level. The bandwidth of LB (BW_{LB}) and RF (BW_{RF}) must accommodate the architecture's single-cycle processing throughput, calculated as $BW_{LB}, BW_{RF} \geq BytesPerCycle \times f$, where f is the PTC operating clock frequency. GLB's bandwidth (BW_{GLB}) is calculated as $BW_{GLB} = MaxLayerSize \cdot f / (N_p \cdot D_p \cdot M_p)$, where the partitioned matrix dimensions are N_p , D_p , and M_p . We first profile the **maximum bandwidth requirement ($\widehat{BW}$) for all sub-architectures** based on the GLB demand per cycle, considering data sharing and broadcast in a specific dataflow. To enable full utilization of the computing cores *without memory bottleneck*, we adopt a **SoTA multi-block SRAM design to meet the bandwidth demand** — A multi-block GLB can boost its bandwidth proportional to the number of blocks [4], [17].

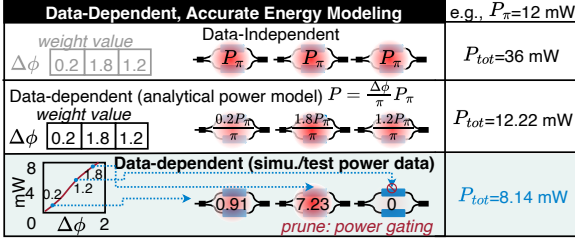


Fig. 5: SimPhony-Sim supports accurate, data-dependent energy modeling with simulated/measured device power data.

SimPhony-Sim automatically searches for the minimum number of required GLB blocks, $\# \text{ of Blocks} = \tau_{GLB} \cdot \widehat{BW} / (b_{bus} \cdot 8)$, where τ_{GLB} is the fastest cycle simulated by CACTI [32], and b_{bus} is the buswidth in bits.

4) *Link Budget Analysis*: Link budget analysis is important for photonic systems to profile the critical-path insertion loss and derive the laser source power requirement and optical signal-to-noise ratio (SNR). To obtain the IL on the critical path, we leverage our constructed hierarchical, weighted DAG representation of the architecture in Fig. 3(a), 3(b). From a given source node, i.e., laser, to a destination node, i.e., photodetector (PD), we will use the graph's longest path to get the critical path IL. Given the PD sensitivity S , to differentiate b_{in} -bit input levels with a target bit error rate, we can derive the lowest laser power required [3], [4],

$$P_{laser} = \frac{10^{(S+IL)/10} \cdot 2^{b_{in}}}{\eta_{WPE}} \cdot \frac{1}{1 - 0.1^{ER/10}}, \quad (1)$$

where η_{WPE} is the laser wall plug efficiency. Non-ideal modulation extinction ratio ER is also considered a power penalty to recover the full modulation range [20].

5) *Data-Dependent Device-Response-Aware Energy Analysis*: To accurately model the energy of EPIC AI systems, SimPhony-Sim captures data access energy via CACTI-simulated memory energy and computing energy based on *actual operand values and realistic device power modeling*.

For data access cost, we derive the memory access size (D_{mem}) for off-chip HBM, GLB, LB, and RF based on the dataflow analysis and accumulate the energy cost with CACTI-simulated access energy per bit e_{mem} , i.e., $E_{mem} = \sum_{mem \in \{HBM, GLB, LB, RF\}} e_{mem} D_{mem}$.

For computing cost, SimPhony-Sim supports **data-dependent energy analysis with accurate device power modeling**, shown in Fig. 5. For analog hardware, the encoded data determines the device configuration, significantly impacting its power. Hence, SimPhony **accumulates the energy over cycles based on the values of the real operands**. This approach enables accurate energy profiling with *fine-grained power gating from ONN pruning* [14]. Device power modeling is also critical. Default library power references, like P_{π} for phase shifters (PSs), often overestimate actual power, while analytical models can be overly ideal. As shown in Fig. 5, SimPhony-Sim supports **customized power models** using analytical, simulation, or chip testing data for power estimation with different fidelity.

6) *Layout-Aware Chip Area Analysis*: The chip area is crucial for fabrication and packaging costs and guiding design optimization. SimPhony-Sim supports fast, realistic layout-aware area estimation. Unlike previous methods that simply sum all

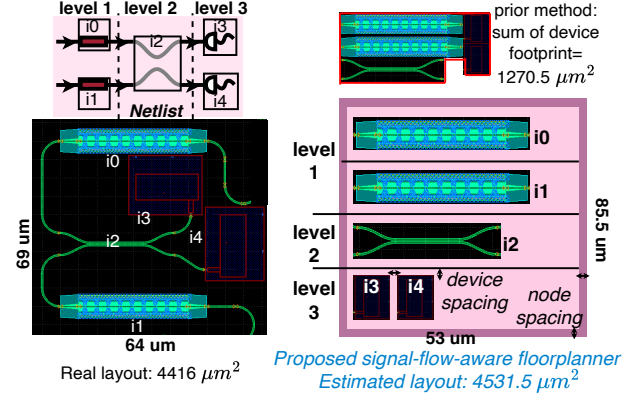


Fig. 6: Proposed signal flow-aware row-based floorplan for accurate layout-aware area estimation.

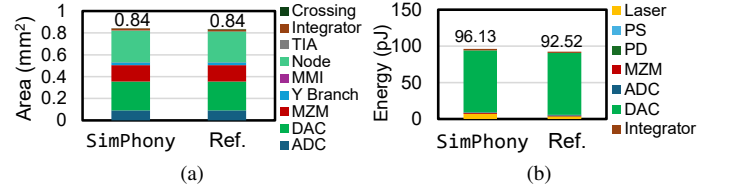


Fig. 7: SimPhony results validated on a $(280 \times 28) \times (28 \times 280)$ GEMM task w/ TeMPO [17]. (a) Area and (b) energy breakdown.

device footprints, SimPhony-Sim either takes in a user-defined bounding box or **automatically generates a signal-flow-aware floorplan**, as shown in Fig. 6. SimPhony sets the placement site width to fit the longest device and attempts to *hide* other devices beneath it. The **floorplan follows the device's topological order** from the netlist to adhere to the minimum bending rule in PIC placement, accounting for user-defined device/node spacing. This approach closely matches the real layout area and can be potentially extended to interface with PIC placement tools.

IV. EVALUATION RESULTS

A. SimPhony Validation

We validate our simulation results by comparing them with architectural evaluation results reported in previous work. It is important to clarify that, in the absence of system-level experimental demonstrations of multi-core photonic AI chips, we compare our results with prior architecture simulation results. We re-emphasize that individual *component data used are backed by experimental measurements*, and area estimates are based on real chip layouts. The functionality and accuracy of the ONN model are ensured by the TorchONN training framework, which is beyond the scope of our architecture performance modeling tool.

GEMM Workloads: To validate the simulation results of SimPhony for GEMM, we compare the simulated area and energy metrics on a $(280 \times 28) \times (28 \times 280)$ GEMM task against the reference values reported in the original TeMPO paper [17] in Fig. 7 with the following architecture settings. We set the core width and height to 4 and the number of tiles and cores per tile to 2. The area 7(a) and energy 7(b) breakdown from SimPhony match the reference results from TeMPO paper.

Dynamic Transformer Workloads: For Transformers with self-attention operations, we validate SimPhony's area and energy results against Lightning-Transformer [4] (LT), shown in Fig. 8.

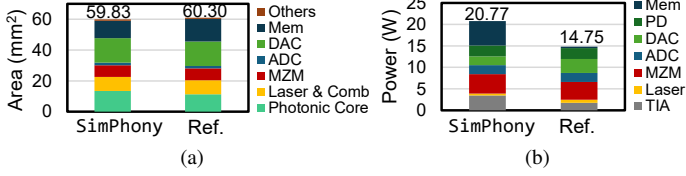


Fig. 8: SimPhony results validated on BERT-Base(ImageNet) w/ Lightning-Transformer [4]. (a) Area and (b) power breakdown.

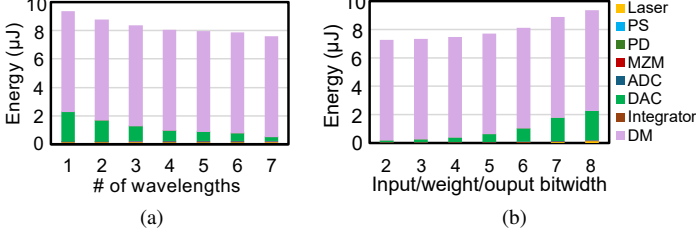


Fig. 9: Sweep (a) # of wavelengths (b) bitwidth on TeMPO arch [17] and $(280 \times 28) \times (28 \times 280)$ GEMM. DM: data movement.

we simulate BERT-Base [33] with a single (224×224) ImageNet-1K [34] image. We adopt LT’s settings: 4 tiles, 2 cores per tile, each core sized 12×12 , with 12 wavelengths operating at 5 GHz. Since LT provides only power breakdowns, we report power instead of energy. SimPhony accurately reproduces the chip area when appropriate scaling factors are applied, matching LT’s reported values. Minor deviations in photonic core and memory areas stem from differences in core area calculations, memory simulations, and device spacing. Our layout-aware estimator effectively generates realistic core areas. Device power aligns with LT’s breakdown except for memory, where deviations result from different technology nodes used in memory simulation (SimPhony uses CACTI-45 nm [32]; LT uses PACTI-14 nm [35]) and SRAM port/bus differences.

B. SimPhony Use Cases

To show the capability of SimPhony, we study multiple design examples by sweeping PTC architectural parameters in SimPhony to gain design insights and demonstrate their impacts on system performance and efficiency. All simulations discussed below have a 4×4 core size with 2 tiles and 2 cores per tile.

1) *Multi-Wavelength Parallelism*: As discussed in Section III-C2, optical systems allow parallelism beyond spatial and temporal dimensions. However, data encoding in these dimensions requires extra power. Figure 9(a) examines TeMPO [17] under varying wavelength settings on a $(280 \times 28) \times (28 \times 280)$ GEMM task while scaling MZM and laser sources with the number of wavelengths. Increased wavelengths enhance parallelism, speeding up computation and reducing energy for components that do not scale with wavelength. However, the MZM’s energy remains constant as the number of MZMs scale with # of wavelengths.

2) *Bitwidth Representation vs. Energy Consumption*: To investigate how the ADC/DAC bit precision impacts the system power, in Fig. 9(b), we sweep the energy for different tensor bitwidths. The results show a clear trend of increasing energy with higher bits. Users can leverage the bitwidth simulation results to find the sweet spot for optimal efficiency.

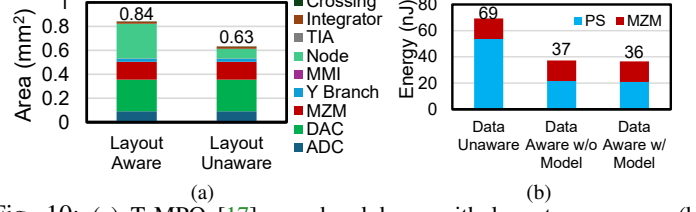


Fig. 10: (a) TeMPO [17] area breakdown with layout awareness. (b) SCATTER [14] energy breakdown with data awareness.

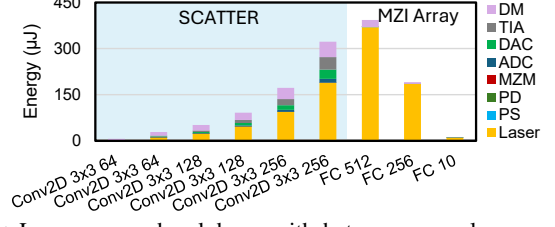


Fig. 11: Layer energy breakdown with heterogeneous layer mapping of VGG-8(CIFAR10) [36]. Convolutions are mapped to SCATTER [14], and Linear layers are mapped to MZI meshes [1].

3) *Layout-Aware and Data-Dependent Modeling*: The exploration of layout-aware area estimation and data-dependent modeling is shown in Fig. 10. The layout-unaware method underestimates the node area by 72%, while our floorplan estimation enables accurate area estimation compared to the real layout. In data-awareness evaluation, we focus on a weight-static PTC SCATTER [14] where weight values impact the phase shifter’s power. By counting PS power based on real weight values, the PS energy decreases from $0.0537 \mu\text{J}$ to $0.0215 \mu\text{J}$ with an approximate power model. If a rigorous device power model is used, the energy is further reduced to $0.0209 \mu\text{J}$ with a substantial 60% reduction.

4) *Heterogeneous Mapping*: Lastly, we show SimPhony’s capability for heterogeneous architecture modeling with layer-to-sub-architecture mapping in Fig. 11. The convolutional layers of VGG-8 are mapped to SCATTER [14], while the linear layers are mapped to MZI meshes [1], while two sub-architectures share the same on-chip memory hierarchy. This hybrid architecture modeling showcases SimPhony’s flexibility in integrating hybrid systems and enabling fine-grained workload-to-hardware mapping. As a future extension, SimPhony can be extended to enable automated design space exploration that combines the strengths of different photonic computing architectures in heterogeneous systems for diverse AI workloads.

V. CONCLUSION

This paper presents SimPhony, a cross-layer modeling and simulation framework for EPIC AI hardware, enabling photonic-specific design evaluation and fair comparisons across implementations. By emphasizing accurate device/circuit-level modeling with generic and extensible representations, SimPhony bridges hardware and software stacks to support flexible hardware construction, validation, and architecture exploration with multi-dimensional metric trade-offs. We have validated SimPhony’s accuracy against prior work and will continue expanding its capabilities to help researchers uncover insights and drive innovation in high-performance, energy-efficient photonic computing systems.

REFERENCES

- [1] Y. Shen, N. C. Harris, S. Skirlo *et al.*, “Deep learning with coherent nanophotonic circuits,” *Nature Photonics*, 2017.
- [2] J. Feldmann, N. Youngblood, M. Karpov, H. Gehring, X. Li, M. Stappers, M. L. Gallo, X. Fu, A. Lukashchuk, A. Raja, J. Liu, D. Wright, A. Sebastian, T. Kippenberg, W. Pernice, and H. Bhaskaran, “Parallel convolutional processing using an integrated photonic tensor core,” *Nature*, 2021.
- [3] C. Feng, J. Gu, H. Zhu, Z. Ying, Z. Zhao *et al.*, “A compact butterfly-style silicon photonic–electronic neural chip for hardware-efficient deep learning,” *ACS Photonics*, vol. 9, no. 12, pp. 3906–3916, 2022.
- [4] H. Zhu, J. Gu, H. Wang, Z. Jiang, Z. Zhang, R. Tang, C. Feng, S. Han *et al.*, “Lightening-transformer: A dynamically-operated photonic tensor core for energy-efficient transformer accelerator,” in *Proc. HPCA*, 2024.
- [5] K. Shiflett, A. Karanth, R. Bunesco, and A. Louri, “Albireo: Energy-efficient acceleration of convolutional neural networks via silicon photonics,” in *Proc. ISCA*, 2021, pp. 860–873.
- [6] B. J. Shastri, A. N. Tait *et al.*, “Photonics for Artificial Intelligence and Neuromorphic Computing,” *Nature Photonics*, 2021.
- [7] Z. Xu, T. Zhou, M. Ma, C. Deng, Q. Dai, and L. Fang, “Large-scale photonic chiplet taichi empowers 160-tops/w artificial general intelligence,” *Science*, vol. 384, no. 6692, pp. 202–209, 2024.
- [8] S. Ning, H. Zhu, C. Feng, J. Gu, Z. Jiang, Z. Ying, J. Midkiff, S. Jain, M. H. Hlaing, D. Z. Pan, and R. T. Chen, “Photonic-electronic integrated circuits for high-performance computing and ai accelerators,” *Journal of Lightwave Technology*, pp. 1–26, 2024.
- [9] A. R. Totović, G. Dabos, N. Passalis, A. Tefas, and N. Pleros, “Femtojoule per MAC Neuromorphic Photonics: An Energy and Technology Roadmap,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 26, no. 5, pp. 1–15, 2020.
- [10] J. Gu, Z. Zhao, C. Feng *et al.*, “Towards Hardware-Efficient Optical Neural Networks: Beyond FFT Architecture via Joint Learnability,” *IEEE TCAD*, 2020.
- [11] J. Gu, Z. Zhao, C. Feng, W. Li, R. T. Chen, and D. Z. Pan, “FLOPS: Efficient On-Chip Learning for Optical Neural Networks Through Stochastic Zeroth-Order Optimization,” in *Proc. DAC*, 2020.
- [12] J. Gu, Z. Zhao, C. Feng, H. Zhu, R. T. Chen, and D. Z. Pan, “ROQ: A noise-aware quantization scheme towards robust optical neural networks with low-bit controls,” in *Proc. DATE*, 2020.
- [13] S. Banerjee, M. Nikdast, S. Pasricha, and K. Chakrabarty, “Pruning Coherent Integrated Photonic Neural Networks Using the Lottery Ticket Hypothesis,” in *2022 IEEE Comput. Soc. Annu. Symp. VLSI ISVLSI*, Jul. 2022, pp. 128–133.
- [14] Z. Yin, N. Gangi, M. Zhang, J. Zhang, R. Huang, and J. Gu, “Scatter: Algorithm-circuit co-sparse photonic accelerator with thermal-tolerant, power-efficient in-situ light redistribution,” in *International Conference on Computer-Aided Design (ICCAD)*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.05510>
- [15] W. Liu, W. Liu, Y. Ye, Q. Lou, Y. Xie, and L. Jiang, “Holylight: A nanophotonic accelerator for deep learning in data centers,” in *Proc. DATE*, 2019.
- [16] F. Zokaee, Q. Lou, N. Youngblood *et al.*, “LightBulb: A Photonic-Nonvolatile-Memory-based Accelerator for Binarized Convolutional Neural Networks,” in *Proc. DATE*, 2020.
- [17] M. Zhang, D. Yin, N. Gangi, A. Begović, A. Chen, Z. R. Huang, and J. Gu, “Tempo: Efficient time-multiplexed dynamic photonic tensor core for edge ai with compact slow-light electro-optic modulator,” *J. Appl. Phys.*, 2024.
- [18] K. Shiflett, D. Wright, A. Karanth, and A. Louri, “PIXEL: Photonic Neural Network Accelerator,” in *Proc. HPCA*, 2020, pp. 474–487.
- [19] C. Demirkiran, G. Yang, D. Bunandar, and A. Joshi, “Mirage: An rns-based photonic accelerator for dnn training,” in *Proc. ISCA*, 2024, pp. 73–87.
- [20] A. N. Tait, T. F. de Lima, E. Zhou *et al.*, “Neuromorphic photonic networks using silicon photonic weight banks,” *Sci. Rep.*, 2017.
- [21] M. Reck, A. Zeilinger, H. Bernstein *et al.*, “Experimental realization of any discrete unitary operator,” *Physical review letters*, 1994.
- [22] W. R. Clements, P. C. Humphreys, B. J. Metcalf *et al.*, “Optimal Design for Universal Multipoint Interferometers,” *Optica*, 2018.
- [23] X. Xu, M. Tan, B. Corcoran, J. Wu, A. Boes, T. G. Nguyen, S. T. Chu, B. E. Little, D. G. Hicks, R. Morandotti, A. Mitchell, and D. J. Moss, “11 TOPS photonic convolutional accelerator for optical neural networks,” *Nature*, 2021.
- [24] T. Andrulis, G. I. Chaudhry, V. M. Suriyakumar, J. S. Emer, and V. Sze, “Architecture-level modeling of photonic deep neural network accelerators,” *arXiv preprint arXiv:2405.07266*, 2024.
- [25] J. Gu, Z. Zhao, C. Feng *et al.*, “Towards area-efficient optical neural networks: an FFT-based architecture,” in *Proc. ASPDAC*, 2020.
- [26] J. Gu, H. Zhu, C. Feng, Z. Jiang, R. T. Chen, and D. Z. Pan, “M3ICRO: Machine Learning-Enabled Compact Photonic Tensor Core based on PRO-grammable Multi-Operand Multimode Interference,” *APL Machine Learning*, 2023.
- [27] M. Miscuglio and V. J. Sorger, “Photonic Tensor Cores for Machine Learning,” *Applied Physics Review*, 2020.
- [28] J. Gu, H. Zhu, C. Feng, Z. Jiang, M. Liu, S. Zhang, R. T. Chen, and D. Z. Pan, “Adept: Automatic differentiable design of photonic tensor cores,” in *Proc. DAC*, 2022.
- [29] M. Li, Z. Yu, Y. Zhang, Y. Fu, and Y. Lin, “O-has: Optical hardware accelerator search for boosting both acceleration performance and development speed,” in *Proc. ICCAD*, 2021.
- [30] J. Gu, H. Zhu, C. Feng, Z. Jiang, R. T. Chen, and D. Z. Pan, “L2ight: Enabling On-Chip Learning for Optical Neural Networks via Efficient in-situ Subspace Optimization,” in *Proc. NeurIPS*, 2021.
- [31] M. Khani, M. Ghobadi, M. Alizadeh, Z. Zhu, M. Glick, K. Bergman, A. Vahdat, B. Klenk, and E. Ebrahimi, “Sip-ml: high-bandwidth optical network interconnects for machine learning training,” in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, ser. SIGCOMM ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 657–675. [Online]. Available: <https://doi.org/10.1145/3452296.3472900>
- [32] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “Cacti 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, jun 2017.
- [33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [34] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *Proc. CVPR*, 2009, pp. 248–255.
- [35] A. Shafaei, Y. Wang, X. Lin, and M. Pedram, “Fincacti: Architectural analysis and modeling of caches with deeply-scaled finfet devices,” in *2014 IEEE Computer Society Annual Symposium on VLSI*, 2014, pp. 290–295.
- [36] N. Simon, J. Friedman, T. Hastie, and R. Tibshirani, “A sparse-group lasso,” *Journal of Computational and Graphical Statistics*, 2013.